The properties we collect data on are as follows:

1. **Per-token losses** with **unaltered models**
2. **Per-token losses** with **ablations** of individual attention heads
3. **Empirical head measurements** with **head activation evaluators:** using heuristics to score attention heads based on their observed activations, as another way of estimating "copying", "prefix matching" and "previous token" attention.
4. **Mathematical head measurements** with the **trace of QK eigenvalues** for the previous-token QK-circuit term: measuring how much the WQWK weight matrix reads in a subspace affected by heads that attend to the previous token.

(1) Per-token losses (unaltered models)

The most straightforward property we measure is per-token losses from each model snapshot. We run the model snapshot on a set of 10,000 randomly-selected examples from the dataset, each of which is 512 tokens in length, and save individual token losses for every token. The set is held the same between all snapshots and between all models in all three families.

When analyzing the per-token loss data, we summarize it in two different ways:

- **Random token per example.** For each example, we extract the loss for just one random token out of the 512 tokens and discard the rest, giving a length-10,000 vector for each model snapshot. (The random index used for each example is consistent between each model.)
- **Context index average.** We average the per-token losses by context index (i.e. by position), giving a length-512 vector for each model snapshot. The *icontext*th entry in the vector is an average over the 10,000 tokens drawn from position *icontext* in each example.

(2) Per-token losses (attention head ablations)

One way to study individual attention heads is to measure the difference between the complete model's overall behavior and the overall behavior of a model that is exactly the same except for missing that head. We do this by successively ablating (i.e. zeroing-out) each attention head's contribution to the residual stream. The approach of removing a head's contribution can be thought of as loosely similar to a genetic "knockout" experiment in biology, where observing the effect of removing a specific gene can provide information about the gene's function.

We could do these ablations in two different ways: a "full" ablation, and a "pattern-preserving" ablation. (The model analysis table shows only pattern-preserving ablations; we explain the other method for comparison) The full ablation would remove *all* of a head's contributions to model behavior, whereas the pattern-preserving ablation removes the ablated head's contribution to all later heads' *output vectors*, but preserves the ablated head's contribution to all later heads' attention *patterns*.[29]

Full ablation

- *Method*: Run the model, replacing the ablated attention head's result vector with a zero vector.
- *Outcome*: All downstream computations are impacted (including all Q, K, and V calculations, and attention patterns).

Pattern-preserving ablation (shown in Model Analysis Table)

- *Method:*
    - Step 1: Run the model for the first time, saving all attention patterns and discarding the output logits.
    - Step 2: Run model a second time. Replace the ablated attention head's result vector with a zero vector, and force all attention patterns to be the version recorded in the first run.
- *Outcome*: Only downstream V calculations are affected by the ablation, not Q and K calculations. The contribution a head makes to later layers' attention patterns is preserved.[30]

This is by far the most computationally-intensive measurement we perform, although fortunately it consists of many small single-GPU jobs and so we could often run it in "spare" capacity.

As a quick note of comparison, our previous work in Transformer Circuits also contained ablation experiments. Although there are similarities (most notably the decision to freeze patterns), the specific ablations we describe here are ***individual attention head*** ablations where only one attention head is modified at a time, whereas the ablations described in our prior work are ***nth-order term*** ablations in which all attention heads are manipulated at once and the process is repeated iteratively *n* times.

(3) Head activation evaluators

We construct three heuristic measurements of attention heads, to score how strongly a head exhibits specific properties of interest. These are empirical measurements, based on observed activations on example data.

We evaluate heads on the following properties:

- **Copying.** Does the head's direct effect on the residual stream increase the logits of the same token as the one being attended to?
- **Prefix matching.** On repeated sequences of random tokens, does the head attend to earlier tokens that are followed by a token that matches the present token?
- **Previous token attention**. Does the head attend to the token that immediately precedes the present token?

(Note that "copying" is a property that depends on a head's OV circuit, whereas "prefix matching" and "previous token attention" are properties that depend on a head's QK circuit.)

We are interested in these properties in particular — copying, prefix matching, and previous token attention — because they are relevant to induction heads' function of predicting repeated sequences of random tokens. Specifically, (1) a head with both a "copying" OV circuit and a "prefix matching" QK circuit is thereby an induction head by our definition, because it attends to previous instances of the present token and increases the logit of the token that followed; and (2) a "previous token" head can be used as an algorithmic component of a future induction head in a later layer (as we explain in more detail in Argument 5).

In more detail, each head evaluator is implemented as follows, and each score is averaged over 10 examples.

- *Copying*: Generate a sequence of 25 random tokens, excluding the most common and the least common tokens. Compute this head's contribution to the residual stream, then convert that using the unembeddings (i.e. along the "direct path") to impacts on each logit. Logits are transformed by subtracting the mean of the logits and passing through a ReLU, allowing the evaluator to focus on where logits are raised. Compute the ratio of the amount it raises the logits of the token being attended to, to that of all tokens in this sample. This value ranges from 0 (only raises other tokens) to 0.5 (only raises the present token), so we scale it into the range of -1 to 1.
- *Prefix matching:* Generate a sequence of 25 random tokens, excluding the most common and the least common tokens. Repeat this sequence 4 times and prepend a "start of sequence" token.[31] Compute the attention pattern. The prefix matching score is the average of all attention pattern entries attending from a given token back to the tokens that preceded the same token in earlier repeats.
- *Previous token:* Draw an example from the training distribution. Compute the attention pattern. The previous token score is the average of the entries of all the attention pattern entries attending from token *i* to token *i-1*.

In the Appendix we validate the connection between these empirical measurements and the mathematical properties of attention head weight matrices explored in Transformer Circuits, and show a joint distribution of scores for copying and prefix matching, for heads in the small and large models.

(4) Trace of QK eigenvalues (for previous-token QK-circuit term)

In addition to the heuristic measurements described above, we construct another measure of **prefix matching** that draws on concepts from A Mathematical Framework for Transformer Circuits. In short, we measure how overall large and positive the eigenvalues are of the W matrix found in the term of the QK circuit that corresponds specifically to "pure K-composition" with heads that attend to the previous token. Positive eigenvalues of W indicate "same-matching": preferring that this component of the query and key vectors be the same. A "same-matching" W matrix associated with a "previous token" term in the QK circuit is a simple way to implement basic prefix matching behavior. We unpack here more about what this measure is and how it is computed.

The "QK circuit" is a term we coined in our previous work to refer specifically to the entire transformation C_{QK}^h which is applied to the one-hot token vectors t to compute the attention pattern A^h for a specific head: $A^h = \text{softmax}^*\left(t^T \cdot C_{QK}^h t\right)$. The full computation of C_{QK}^h for a given head can be decomposed into many terms, each corresponding to one of the many paths through the model that needs to be computed in order to map tokens to their attention scores for that head. The section Path Expansion of Attention Scores QK Circuit in our earlier work shows all the terms of a full expansion for a two-layer model; models with more layers have QK circuits with even more terms.

For this measure, we narrow the scope of our calculation to the simplest possible term of the QK circuit that could contribute to the "prefix-matching" behavior of an induction head: namely, only pure K-composition terms (in which the query-side term is just the direct path from the embeddings), and furthermore only the subset of those that correspond specifically to K-composing with an earlier head that consistently attends to the immediately previous token.

In order to compute this measure for a given head, we do the following:

1. For all attention heads in earlier layers, measure how "previous-token-like" the head is, as the average of the attention pattern along the previous-token off-diagonal, over an example text.
2. Compute the W matrix found in each individual $Id \otimes A^{h_k} \otimes W_E^T W_{QK}^h W_{OV}^{h_k} W_E$ term, for each head in earlier layers.
3. Create a combined W matrix by weighting the matrices in step 2 by the measurement in step 1 of how "previous-token-like" each previous head is.
4. Measure the trace (i.e. the sum of the eigenvalues) of this combined weighted W matrix.

Because this measure relies on K-composition with heads in previous layers, it is only defined for models more than one layer deep, and only for attention heads in the second layer and beyond.

We show in the Appendix that this measure correlates well with our heuristic prefix matching evaluator on our small models.

Analyses

Per-token losses

The second through fourth rows of the Model Analysis Table show results derived from the per-token losses from the small models and full-scale models. We show the following "views" on the per-token loss data:

- **Loss curve**: The *random-token-per-example* loss data — a matrix of size (N model snapshots x 10,000 examples) for N = 200 snapshots per small model and N = 15 snapshots per full-scale model — is averaged over the example dimension, giving an average loss per model snapshot.
- **2D in-context learning plot**: The *context-index-average* loss data — a matrix of size (N model snapshots x 512 token index positions) — is visualized directly as a 2D plot. This shows model performance on tokens earlier versus later in the context, as well as across training time. We borrow this technique for highlighting "in-context learning" from Kaplan *et al.* [13] . A horizontal slice of this plot represents a loss curve as a function of training step, for one particular token index rather than averaged over all tokens; a vertical slice represents a particular snapshot's prediction ability as a function of token index.
- **2D loss derivative plot:** We also plot the partial derivative of the context-index-average loss data with respect to log(context index). This captures the amount that token loss is reduced by seeing ϵ% more context.
- **In-context learning score**: We compute what we call the "in-context learning score" as the average loss of the 500th token minus the average loss of the 50th token (for a length-512 context)± over training. This functions as a summary of the 2D in-context learning plot, capturing in a single summary statistic the trajectory over training time of the model's ability to make better predictions later in the context than earlier.